



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Dianna Francesca M. Pardilla Justin D. Vergara
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	Oct 3, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)

- Google Drive for shared documentation
- GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
(Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF file**.

Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

- ✓ Similarity score or ranking
- ✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

- ✓ “New case saved” or similar confirmation
 - ✓ Updated case list
-

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas*.
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system’s knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.

System retrieves past students with similar error patterns (e.g., misusing formulas).

It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks



```
from dataclasses import dataclass
from typing import List, Tuple, Optional
import time

# ANSI PALETTE & UI STYLING ENGINE
class Style:
    RESET    = "\033[0m"
    BOLD      = "\033[1m"
    DIM       = "\033[2m"

    # Soft Pastel Accents
    CYAN      = "\033[38;2;110;207;246m"
    LILAC     = "\033[38;2;199;146;234m"
    MINT      = "\033[38;2;137;221;110m"
    CORAL     = "\033[38;2;240;113;120m"
    GOLD      = "\033[38;2;255;203;107m"
    SLATE     = "\033[38;2;120;144;156m"
    WHITE     = "\033[38;2;238;255;255m"

def draw_header(title: str, subtitle: str, programmers: List[str],
section: str) -> None:
    width = 72
    print(f"\n{Style.LILAC}{ '-' * (width - 2) }{Style.RESET}")
    print(f"{Style.LILAC}|{Style.BOLD}{Style.WHITE} {title.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
    print(f"{Style.LILAC}|{Style.DIM}{Style.CYAN} {subtitle.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
    print(f"{Style.LILAC}|{ '-' * (width - 2) }{Style.RESET}")

    prog_line = f"Programmed by: {' '.join(programmers)}"
    sec_line = f"Course & Section: {section}"
    print(f"{Style.LILAC}|{Style.SLATE} {prog_line.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
    print(f"{Style.LILAC}|{Style.SLATE} {sec_line.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
    print(f"{Style.LILAC}^{ '-' * (width - 2) }^{Style.RESET}")
```

```

def draw_section(step_num: int, name: str, badge: str = "") -> None:
    badge_str = f" {Style.DIM}[{badge}]{Style.RESET}" if badge else ""
    print(f"\n{Style.BOLD}{Style.GOLD}► STEP {step_num}:{Style.RESET}"
          {Style.BOLD}{Style.WHITE}{name}{Style.RESET}{badge_str}")
    print(f"{Style.SLATE} {'-' * 64}{Style.RESET}")

def render_meter(score: float, width: int = 18) -> str:
    filled = int(round(score * width))
    bar = "■" * filled + "░" * (width - filled)
    return f"{Style.MINT}{bar}{Style.RESET} {Style.BOLD}{score * 100:>5.1f}%{Style.RESET}"

# DOMAIN MODELS (Data Structures)
@dataclass
class ProblemState:
    topic: str
    error_type: str
    proficiency_level: str
    error_frequency: int

@dataclass
class RemedialIntervention:
    intervention: str
    recommended_practice: str
    difficulty_adjustment: str

@dataclass
class TutoringCase:
    case_id: str
    problem: ProblemState
    solution: RemedialIntervention

# 1. INITIAL CASE BASE
case_base: List[TutoringCase] = [
    TutoringCase(
        case_id="CASE-001",

```



```

        problem=ProblemState("Algebra", "Sign Error", "Beginner", 4),
        solution=RemedialIntervention(
            "Review integer sign rules with interactive number-line
drills.",
            "5 foundational integer arithmetic exercises",
            "Decrease difficulty"
        )
    ),
    TutoringCase(
        case_id="CASE-002",
        problem=ProblemState("Algebra", "Formula Misuse", "Intermediate",
3),
        solution=RemedialIntervention(
            "Step-by-step formula breakdown focusing on variable
substitution.",
            "3 guided formula substitution problems",
            "Maintain current difficulty"
        )
    ),
    TutoringCase(
        case_id="CASE-003",
        problem=ProblemState("Calculus", "Chain Rule Omission",
"Advanced", 2),
        solution=RemedialIntervention(
            "Visual breakdown of nested functions and outer/inner
derivative scaffolding.",
            "4 multi-step chain rule problems",
            "Maintain current difficulty"
        )
    ),
    TutoringCase(
        case_id="CASE-004",
        problem=ProblemState("Geometry", "Unit Mismatch", "Beginner", 5),
        solution=RemedialIntervention(
            "Unit conversion refresher focusing on metric dimension
alignment.",
            "6 dimensional analysis drills",

```

```

        "Decrease difficulty"
    )
),
TutoringCase(
    case_id="CASE-005",
    problem=ProblemState("Algebra", "Order of Operations",
"Intermediate", 2),
    solution=RemedialIntervention(
        "PEMDAS sequence reinforcement using color-coded grouping
brackets.",
        "4 precedence-focused evaluations",
        "Maintain current difficulty"
    )
),
]

# 2. SIMILARITY ASSESSMENT (Retrieve Stage)
def calculate_similarity(p1: ProblemState, p2: ProblemState) ->
Tuple[float, dict]:
    weights = {"topic": 0.30, "error": 0.40, "proficiency": 0.15,
"frequency": 0.15}

    s_topic = 1.0 if p1.topic.lower() == p2.topic.lower() else 0.0
    s_error = 1.0 if p1.error_type.lower() == p2.error_type.lower() else
0.0
    s_prof = 1.0 if p1.proficiency_level.lower() ==
p2.proficiency_level.lower() else 0.0

    freq_delta = abs(p1.error_frequency - p2.error_frequency)
    s_freq = max(0.0, 1.0 - (freq_delta / 5.0))

    total = (
        weights["topic"] * s_topic +
        weights["error"] * s_error +
        weights["proficiency"] * s_prof +
        weights["frequency"] * s_freq
    )

```

```

breakdown = {
    "Topic Match": s_topic,
    "Error Match": s_error,
    "Level Match": s_prof,
    "Freq Proximity": round(s_freq, 2)
}
return round(total, 4), breakdown

def retrieve_best_case(new_problem: ProblemState) -> Tuple[TutoringCase,
float, dict]:
    best_case: Optional[TutoringCase] = None
    best_score = -1.0
    best_breakdown = {}

    for case in case_base:
        score, breakdown = calculate_similarity(new_problem, case.problem)
        if score > best_score:
            best_score = score
            best_case = case
            best_breakdown = breakdown

    return best_case, best_score, best_breakdown

# 3. SOLUTION ADAPTATION (Reuse & Revise Stages)
def adapt_solution(retrieved_case: TutoringCase, new_prob: ProblemState)
-> RemedialIntervention:
    base = retrieved_case.solution
    adapted_action = base.intervention
    adapted_practice = base.recommended_practice
    adapted_diff = base.difficulty_adjustment

    # Rule 1: High repeated error threshold (>= 4 repeats)
    if new_prob.error_frequency >= 4:
        adapted_diff = "Decrease difficulty (Remedial Mode Activated)"
        adapted_practice += " + 2 scaffolded verification drills"

```

```

        elif new_prob.error_frequency <= 1 and
new_prob.proficiency_level.lower() == "advanced":
    adapted_diff = "Increase difficulty (Accelerated Track)"

    # Rule 2: Cross-topic domain adjustment
    if new_prob.topic.lower() != retrieved_case.problem.topic.lower():
        adapted_action += f" [Adapted from {retrieved_case.problem.topic}
to {new_prob.topic}]"

    return RemedialIntervention(
        intervention=adapted_action,
        recommended_practice=adapted_practice,
        difficulty_adjustment=adapted_diff
    )

# 4. KNOWLEDGE RETENTION (Retain Stage)
def retain_case(new_problem: ProblemState, final_solution:
RemedialIntervention) -> str:
    new_id = f"CASE-{len(case_base) + 1:03d}"
    learned_case = TutoringCase(
        case_id=new_id,
        problem=new_problem,
        solution=final_solution
    )
    case_base.append(learned_case)
    return new_id

# 5. EXECUTION PIPELINE
def run_its_cbr_pipeline(target_problem: ProblemState) -> None:
    # Step 1: Input Diagnosis
    draw_section(1, "Learner Diagnosis Input", "Ingestion")
    print(f"      {Style.CYAN}• Topic                                :{Style.RESET}
{target_problem.topic}")
    print(f"      {Style.CYAN}• Detected Flaw                        :{Style.RESET}
{Style.CORAL}{target_problem.error_type}{Style.RESET}")
    print(f"      {Style.CYAN}• Skill Level                            :{Style.RESET}
{target_problem.proficiency_level}")

```

```

        print(f"          {Style.CYAN}•   Error   Frequency   :{Style.RESET}
{target_problem.error_frequency} repeated occurrences")

# Step 2: Retrieval
draw_section(2, "Experiential Retrieval", "Retrieval Phase")
best_case, sim_score, breakdown = retrieve_best_case(target_problem)

        print(f"          {Style.CYAN}•   Best   Match   Case   :{Style.RESET}
{Style.BOLD}{best_case.case_id}{Style.RESET}")
        print(f"          {Style.CYAN}•   Match   Affinity       :{Style.RESET}
{render_meter(sim_score)})")
        print(f"          {Style.SLATE}  └ Breakdown       : {breakdown}{Style.RESET}")
        print(f"          {Style.CYAN}•   Past   Baseline           :{Style.RESET}
\"{best_case.solution.intervention}\"")

# Step 3: Reuse & Revise
draw_section(3, "Solution Adaptation & Revision", "Pedagogical
Tuning")
adapted = adapt_solution(best_case, target_problem)

        print(f"          {Style.MINT}✓   Action   Assigned   :{Style.RESET}
{adapted.intervention}")
        print(f"          {Style.MINT}✓   Practice   Set           :{Style.RESET}
{adapted.recommended_practice}")
        print(f"          {Style.MINT}✓   Dynamic   Pacing       :{Style.RESET}
{Style.GOLD}{adapted.difficulty_adjustment}{Style.RESET}")

# Step 4: Retain
draw_section(4, "Experiential Retention", "Continuous Learning")
new_case_id = retain_case(target_problem, adapted)

        print(f"          {Style.LILAC}◆ Learning State   :{Style.RESET} Preserving
experience into local case memory...")
        print(f"          {Style.LILAC}◆   Stored   Key           :{Style.RESET}
{Style.BOLD}{new_case_id}{Style.RESET}")
        print(f"          {Style.LILAC}◆ Memory Pool           :{Style.RESET} Case Base
incremented to {Style.BOLD}{len(case_base)}{Style.RESET} active entries")

```

```

    print(f"\n{Style.SLATE}{'-' * 72}{Style.RESET}")

# CASE BASE VIEWER
def display_case_base() -> None:
    width = 72
    print(f"\n{Style.CYAN}{ '-' * (width - 2) }{Style.RESET}")
    print(f"{Style.CYAN}|{Style.BOLD}{Style.WHITE} {'CURRENT ITS CASE BASE REPOSITORY'.center(width - 4)} {Style.RESET}{Style.CYAN}|{Style.RESET}")
    print(f"{Style.CYAN}{ '-' * (width - 2) }{Style.RESET}")

    for case in case_base:
        p = case.problem
        s = case.solution

        print(f"\n      {Style.GOLD}[{case.case_id}]{Style.RESET}
{Style.BOLD}{p.topic}{Style.RESET} |
{Style.CORAL}{p.error_type}{Style.RESET}    ({p.proficiency_level},    Freq:
{p.error_frequency})")
        print(f"      {Style.SLATE}Action  :{Style.RESET} {s.intervention}")
        print(f"      {Style.SLATE}Practice:{Style.RESET}
{s.recommended_practice}")
        print(f"      {Style.SLATE}Pacing    :{Style.RESET}
{s.difficulty_adjustment}")

    print(f"\n{Style.SLATE}{'-' * 72}{Style.RESET}")

# INTERACTIVE CLI MAIN MENU
def main_menu():
    while True:
        draw_header(
            title="INTELLIGENT TUTORING SYSTEM (ITS)",
            subtitle="Case-Based Pedagogical Reasoning Engine",
            programmers=["Dianna Francesca M. Pardilla", "Justin D. Vergara"],
            section="CS0065 - AN41"
        )

```

```

        print(f"    {Style.BOLD}[1]{Style.RESET} Run Default Assessment
Scenario")

        print(f"    {Style.BOLD}[2]{Style.RESET} Enter Custom Student Error
(Solve & Retain)")

        print(f"    {Style.BOLD}[3]{Style.RESET} View Entire Case Base
Repository ({len(case_base)} cases)")

        print(f"    {Style.BOLD}[4]{Style.RESET} Exit System")
        print(f"{Style.SLATE}{'-' * 72}{Style.RESET}")

        choice = input(f"{Style.GOLD}Select option [1-4]:
{Style.RESET}").strip()

        if choice == "1":
            default_case = ProblemState(
                topic="Algebra",
                error_type="Formula Misuse",
                proficiency_level="Beginner",
                error_frequency=5
            )
            run_its_cbr_pipeline(default_case)
            input(f"\n{Style.DIM}Press Enter to return to main
menu...{Style.RESET}")

        elif choice == "2":
            print(f"\n{Style.CYAN}--- Enter New Student Data
---{Style.RESET}")
            topic = input("Enter Topic (e.g., Algebra, Geometry,
Calculus): ").strip().capitalize() or "Algebra"
            error = input("Enter Error Type (e.g., Formula Misuse, Sign
Error, Unit Mismatch): ").strip().title() or "Sign Error"

            print("Proficiency Levels: [1] Beginner [2] Intermediate [3]
Advanced")
            lvl_choice = input("Select level (1-3): ").strip()
            level_map = {"1": "Beginner", "2": "Intermediate", "3":
"Advanced"}
            level = level_map.get(lvl_choice, "Beginner")

```

```

        try:
            freq = int(input("Enter Error Frequency (1-10):
").strip())
        except ValueError:
            freq = 3

        custom_problem = ProblemState(topic, error, level, freq)
        run_its_cbr_pipeline(custom_problem)
        input(f"\n{Style.DIM}Press Enter to return to main
menu...{Style.RESET}")

    elif choice == "3":
        display_case_base()
        input(f"\n{Style.DIM}Press Enter to return to main
menu...{Style.RESET}")

    elif choice == "4":
        print(f"\n{Style.MINT}Exiting Intelligent Tutoring System.
Goodbye!{Style.RESET}\n")
        break

    else:
        print(f"{Style.CORAL}Invalid choice! Please select 1, 2, 3, or
4.{Style.RESET}")
        time.sleep(1)

if __name__ == "__main__":
    main_menu()

```



```

its_cbr_system.py X
its_system.py > calculate_similarity
1 from dataclasses import dataclass
2 from typing import List, Tuple, Optional
3 import time
4
5 # ANSI PALETTE & UI STYLING ENGINE
6 class Style:
7     RESET = "\033[0m"
8     BOLD = "\033[1m"
9     DIM = "\033[2m"
10
11     # Soft Pastel Accents
12     CYAN = "\033[38;2;110;207;246m"
13     LILAC = "\033[38;2;199;146;234m"
14     MINT = "\033[38;2;137;221;110m"
15     CORAL = "\033[38;2;240;113;120m"
16     GOLD = "\033[38;2;255;203;107m"
17     SLATE = "\033[38;2;120;144;156m"
18     WHITE = "\033[38;2;238;255;255m"
19
20 def draw_header(title: str, subtitle: str, programmers: List[str], section: str) -> None:
21     width = 72
22     print(f"\n{Style.LILAC}{ '-' * (width - 2) }{Style.RESET}")
23     print(f"{Style.LILAC}|{Style.BOLD}{Style.WHITE} {title.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
24     print(f"{Style.LILAC}|{Style.DIM}{Style.CYAN} {subtitle.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
25     print(f"{Style.LILAC}{ '-' * (width - 2) }{Style.RESET}")
26
27     prog_line = f"Programmed by: {', '.join(programmers)}"
28     sec_line = f"Course & Section: {section}"
29     print(f"{Style.LILAC}|{Style.SLATE} {prog_line.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
30     print(f"{Style.LILAC}|{Style.SLATE} {sec_line.center(width - 4)} {Style.RESET}{Style.LILAC}|{Style.RESET}")
31     print(f"{Style.LILAC}{ '-' * (width - 2) }{Style.RESET}")
32
33 def draw_section(step_num: int, name: str, badge: str = "") -> None:
34     badge_str = f" {Style.DIM}[{badge}]{Style.RESET}" if badge else ""
35     print(f"\n{Style.BOLD}{Style.GOLD}> STEP {step_num}:{Style.RESET} {Style.BOLD}{Style.WHITE}{name}{Style.RESET}{badge_str}")
36     print(f"{Style.SLATE} { '-' * 64 }{Style.RESET}")
37
38 def render_meter(score: float, width: int = 18) -> str:
39     filled = int(round(score * width))
40     bar = "█" * filled + "░" * (width - filled)
41     return f"{Style.MINT}{bar}{Style.RESET} {Style.BOLD}{score * 100:>5.1f}%{Style.RESET}"
42
43 # DOMAIN MODELS (Data Structures)
44 @dataclass
45 class ProblemState:
46     topic: str
47     error_type: str
48     proficiency_level: str

```

```
its_cbr_system.py X
its_cbr_system.py > ...
45 class ProblemState:
46     error_frequency: int
47
48 @dataclass
49 class RemedialIntervention:
50     intervention: str
51     recommended_practice: str
52     difficulty_adjustment: str
53
54 @dataclass
55 class TutoringCase:
56     case_id: str
57     problem: ProblemState
58     solution: RemedialIntervention
59
60 # 1. INITIAL CASE BASE
61 case_base: List[TutoringCase] = [
62     TutoringCase(
63         case_id="CASE-001",
64         problem=ProblemState("Algebra", "Sign Error", "Beginner", 4),
65         solution=RemedialIntervention(
66             "Review integer sign rules with interactive number-line drills.",
67             "5 foundational integer arithmetic exercises",
68             "Decrease difficulty"
69         ),
70     ),
71     TutoringCase(
72         case_id="CASE-002",
73         problem=ProblemState("Algebra", "Formula Misuse", "Intermediate", 3),
74         solution=RemedialIntervention(
75             "Step-by-step formula breakdown focusing on variable substitution.",
76             "3 guided formula substitution problems",
77             "Maintain current difficulty"
78         ),
79     ),
80     TutoringCase(
81         case_id="CASE-003",
82         problem=ProblemState("Calculus", "Chain Rule Omission", "Advanced", 2),
83         solution=RemedialIntervention(
84             "Visual breakdown of nested functions and outer/inner derivative scaffolding.",
85             "4 multi-step chain rule problems",
86             "Maintain current difficulty"
87         ),
88     ),
89     TutoringCase(
90         case_id="CASE-004",
91         problem=ProblemState("Geometry", "Unit Mismatch", "Beginner", 5),
92         solution=RemedialIntervention(
```

```

its_cbr_system.py > ...
    solution=RemedialIntervention(
196         "Unit conversion refresher focusing on metric dimension alignment.",
197         "6 dimensional analysis drills",
198         "Decrease difficulty"
199     )
200 ),
201 TutoringCase(
202     case_id="CASE-005",
203     problem=ProblemState("Algebra", "Order of Operations", "Intermediate", 2),
204     solution=RemedialIntervention(
205         "PEMDAS sequence reinforcement using color-coded grouping brackets.",
206         "4 precedence-focused evaluations",
207         "Maintain current difficulty"
208     )
209 ),
210 ]
211
212 # 2. SIMILARITY ASSESSMENT (Retrieve Stage)
213 def calculate_similarity(p1: ProblemState, p2: ProblemState) -> Tuple[float, dict]:
214     weights = {"topic": 0.30, "error": 0.40, "proficiency": 0.15, "frequency": 0.15}
215
216     s_topic = 1.0 if p1.topic.lower() == p2.topic.lower() else 0.0
217     s_error = 1.0 if p1.error_type.lower() == p2.error_type.lower() else 0.0
218     s_prof = 1.0 if p1.proficiency_level.lower() == p2.proficiency_level.lower() else 0.0
219
220     freq_delta = abs(p1.error_frequency - p2.error_frequency)
221     s_freq = max(0.0, 1.0 - (freq_delta / 5.0))
222
223     total = (
224         weights["topic"] * s_topic +
225         weights["error"] * s_error +
226         weights["proficiency"] * s_prof +
227         weights["frequency"] * s_freq
228     )
229
230     breakdown = {
231         "Topic Match": s_topic,
232         "Error Match": s_error,
233         "Level Match": s_prof,
234         "Freq Proximity": round(s_freq, 2)
235     }
236     return round(total, 4), breakdown
237
238 def retrieve_best_case(new_problem: ProblemState) -> Tuple[TutoringCase, float, dict]:
239     best_case: Optional[TutoringCase] = None
240     best_score = -1.0
241     best_breakdown = {}
242

```

```

143     for case in case_base:
144         score, breakdown = calculate_similarity(new_problem, case.problem)
145         if score > best_score:
146             best_score = score
147             best_case = case
148             best_breakdown = breakdown
149
150     return best_case, best_score, best_breakdown
151
152 # 3. SOLUTION ADAPTATION (Reuse & Revise Stages)
153 def adapt_solution(retrieved_case: TutoringCase, new_prob: ProblemState) -> RemedialIntervention:
154     base = retrieved_case.solution
155     adapted_action = base.intervention
156     adapted_practice = base.recommended_practice
157     adapted_diff = base.difficulty_adjustment
158
159     # Rule 1: High repeated error threshold (>= 4 repeats)
160     if new_prob.error_frequency >= 4:
161         adapted_diff = "Decrease difficulty (Remedial Mode Activated)"
162         adapted_practice += " + 2 scaffolded verification drills"
163     elif new_prob.error_frequency <= 1 and new_prob.proficiency_level.lower() == "advanced":
164         adapted_diff = "Increase difficulty (Accelerated Track)"
165
166     # Rule 2: Cross-topic domain adjustment
167     if new_prob.topic.lower() != retrieved_case.problem.topic.lower():
168         adapted_action += f" [Adapted from {retrieved_case.problem.topic} to {new_prob.topic}]"
169
170     return RemedialIntervention(
171         intervention=adapted_action,
172         recommended_practice=adapted_practice,
173         difficulty_adjustment=adapted_diff
174     )
175
176 # 4. KNOWLEDGE RETENTION (Retain Stage)
177 def retain_case(new_problem: ProblemState, final_solution: RemedialIntervention) -> str:
178     new_id = f"CASE-{len(case_base) + 1:03d}"
179     learned_case = TutoringCase(
180         case_id=new_id,
181         problem=new_problem,
182         solution=final_solution
183     )
184     case_base.append(learned_case)
185     return new_id
186

```

```

its_cbr_system.py X
its_cbr_system.py > retain_case
# 5. EXECUTION PIPELINE
187 def run_its_cbr_pipeline(target_problem: ProblemState) -> None:
188     # Step 1: Input Diagnosis
189     draw_section(1, "Learner Diagnosis Input", "Ingestion")
190     print(f" {Style.CYAN}• Topic           :{Style.RESET} {target_problem.topic}")
191     print(f" {Style.CYAN}• Detected Flaw      :{Style.RESET} {Style.CORAL}{target_problem.error_type}{Style.RESET}")
192     print(f" {Style.CYAN}• Skill Level       :{Style.RESET} {target_problem.proficiency_level}")
193     print(f" {Style.CYAN}• Error Frequency  :{Style.RESET} {target_problem.error_frequency} repeated occurrences")
194
195     # Step 2: Retrieval
196     draw_section(2, "Experiential Retrieval", "Retrieval Phase")
197     best_case, sim_score, breakdown = retrieve_best_case(target_problem)
198
199     print(f" {Style.CYAN}• Best Match Case :{Style.RESET} {Style.BOLD}{best_case.case_id}{Style.RESET}")
200     print(f" {Style.CYAN}• Match Affinity  :{Style.RESET} {render_meter(sim_score)}")
201     print(f" {Style.SLATE}  ↳ Breakdown    :{breakdown}{Style.RESET}")
202     print(f" {Style.CYAN}• Past Baseline   :{Style.RESET} \"{best_case.solution.intervention}\"")
203
204     # Step 3: Reuse & Revise
205     draw_section(3, "Solution Adaptation & Revision", "Pedagogical Tuning")
206     adapted = adapt_solution(best_case, target_problem)
207
208     print(f" {Style.MINT}✓ Action Assigned :{Style.RESET} {adapted.intervention}")
209     print(f" {Style.MINT}✓ Practice Set   :{Style.RESET} {adapted.recommended_practice}")
210     print(f" {Style.MINT}✓ Dynamic Pacing  :{Style.RESET} {Style.GOLD}{adapted.difficulty_adjustment}{Style.RESET}")
211
212     # Step 4: Retain
213     draw_section(4, "Experiential Retention", "Continuous Learning")
214     new_case_id = retain_case(target_problem, adapted)
215
216     print(f" {Style.LILAC}• Learning State :{Style.RESET} Preserving experience into local case memory...")
217     print(f" {Style.LILAC}• Stored Key      :{Style.RESET} {Style.BOLD}{new_case_id}{Style.RESET}")
218     print(f" {Style.LILAC}• Memory Pool    :{Style.RESET} Case Base incremented to {Style.BOLD}{len(case_base)}{Style.RESET} active entries")
219     print(f" {Style.SLATE}{ '-' * 72}{Style.RESET}")
220
221 # CASE BASE VIEWER
222 def display_case_base() -> None:
223     width = 72
224     print(f"\n{Style.CYAN}┌{'-' * (width - 2)}┐{Style.RESET}")
225     print(f" {Style.CYAN}└{Style.BOLD}{Style.WHITE} 'CURRENT ITS CASE BASE REPOSITORY'.center(width - 4)}{Style.RESET}{Style.CYAN}┘{Style.RESET}")
226     print(f" {Style.CYAN}└{'-' * (width - 2)}┘{Style.RESET}")
227
228     for case in case_base:
229         p = case.problem
230         s = case.solution
231         print(f"\n {Style.GOLD}[{case.case_id}]{Style.RESET} {Style.BOLD}{p.topic}{Style.RESET} | {Style.CORAL}{p.error_type}{Style.RESET} ({p.proficiency_level}, Freq: {p.error_frequency})")
232         print(f" {Style.SLATE}Action      :{Style.RESET} {s.intervention}")
233         print(f" {Style.SLATE}Practice:{Style.RESET} {s.recommended_practice}")

```

```

235     print(f"    {Style.SLATE}Pacing    :{Style.RESET} {s.difficulty_adjustment}")
236
237     print(f"\n{Style.SLATE}{'-' * 72}{Style.RESET}")
238
239 # INTERACTIVE CLI MAIN MENU
240 def main_menu():
241     while True:
242         draw_header(
243             title="INTELLIGENT TUTORING SYSTEM (ITS)",
244             subtitle="Case-Based Pedagogical Reasoning Engine",
245             programmers=["Dianna Francesca M. Pardilla", "Justin D. Vergara"],
246             section="CS0065 - AN41"
247         )
248
249         print(f"    {Style.BOLD}[1]{Style.RESET} Run Default Assessment Scenario")
250         print(f"    {Style.BOLD}[2]{Style.RESET} Enter Custom Student Error (Solve & Retain)")
251         print(f"    {Style.BOLD}[3]{Style.RESET} View Entire Case Base Repository ({len(case_base)} cases)")
252         print(f"    {Style.BOLD}[4]{Style.RESET} Exit System")
253         print(f"{Style.SLATE}{'-' * 72}{Style.RESET}")
254
255         choice = input(f"{Style.GOLD}Select option [1-4]: {Style.RESET}").strip()
256
257         if choice == "1":
258             default_case = ProblemState(
259                 topic="Algebra",
260                 error_type="Formula Misuse",
261                 proficiency_level="Beginner",
262                 error_frequency=5
263             )
264             run_its_cbr_pipeline(default_case)
265             input(f"\n{Style.DIM}Press Enter to return to main menu...{Style.RESET}")
266
267         elif choice == "2":
268             print(f"\n{Style.CYAN}--- Enter New Student Data ---{Style.RESET}")
269             topic = input("Enter Topic (e.g., Algebra, Geometry, Calculus): ").strip().capitalize() or "Algebra"
270             error = input("Enter Error Type (e.g., Formula Misuse, Sign Error, Unit Mismatch): ").strip().title() or "Sign Error"
271
272             print("Proficiency Levels: [1] Beginner [2] Intermediate [3] Advanced")
273             lvl_choice = input("Select level (1-3): ").strip()
274             level_map = {"1": "Beginner", "2": "Intermediate", "3": "Advanced"}
275             level = level_map.get(lvl_choice, "Beginner")
276
277             try:
278                 freq = int(input("Enter Error Frequency (1-10): ").strip())

```

```

279 ✓         except ValueError:
280             freq = 3
281
282             custom_problem = ProblemState(topic, error, level, freq)
283             run_its_cbr_pipeline(custom_problem)
284             input(f"\n{Style.DIM}Press Enter to return to main menu...{Style.RESET}")
285
286 ✓         elif choice == "3":
287             display_case_base()
288             input(f"\n{Style.DIM}Press Enter to return to main menu...{Style.RESET}")
289
290 ✓         elif choice == "4":
291             print(f"\n{Style.MINT}Exiting Intelligent Tutoring System. Goodbye!{Style.RESET}\n")
292             break
293
294 ✓         else:
295             print(f"{Style.CORAL}Invalid choice! Please select 1, 2, 3, or 4.{Style.RESET}")
296             time.sleep(1)
297
298 ✓ if __name__ == "__main__":
299     main_menu()

```

CASE STUDY 3: INTELLIGENT TUTORING SYSTEM (ITS)

I. DESCRIPTION

This case study serves as the persistent memory of the Intelligent Tutoring System, that stores the historical student learning challenges alongside the solution used to help them. Each case is saved as a Python dictionary with two parts:

- **Problem:**
 - **Topic:** The math subject the student is studying, such as Algebra, Calculus, or Geometry.
 - **Error Type:** The specific mistake made, such as Formula Misuse, Sign Error, or Unit Mismatch.
 - **Proficiency Level:** The skill level of the student (Beginner, Intermediate, or Advanced.)
 - **Error Frequency:** How often the student repeated the mistake.
- **Solution:**
 - **Difficulty Adjustment:** The lesson or guidance provided to address the conceptual mistake.
 - **Intervention:** The specific set of practice problems assigned to the student.
 - **Recommended Practice:** The pacing change applied, such as decreasing, maintaining, or increasing the task difficulty.

II. SIMILARITY METHOD

The system compares the new and old student's problem cases using a weighted score:

- **Error Type Match (40% weight):** Highest weight because the system must target the exact mistake the student made.
- **Topic Match (30% weight):** Checks if both problems belong to the same math subject.
- **Proficiency Level Match (15% weight):** Checks if both students, old and new, share the same learning baseline.
- **Error Frequency Proximity (15% weight):** Checks how close the number of repeated mistakes is between the two cases.

The program then calculates the values, sums it up, and retrieves the past cases with the highest similarity score.

III. ADAPTATION LOGIC

The system adjusts the retrieved past solution based on the current student's needs instead of copying the solution directly:

- **High Repeated Mistakes:** If the student repeats a mistake 3 or more times, the system lowers the difficulty to remedial mode and adds 2 extra practice questions.
- **High Skill with Low Errors:** If an Advanced student makes 1 or 2 mistakes, the system increases the difficulty for accelerated learning.
- **Topic Change:** If the best past case comes from a different math topic, the system takes note of the lesson that was adapted to the current subject.

IV. SCREENSHOTS OF PROGRAM EXECUTION

```
INTELLIGENT TUTORING SYSTEM (ITS)
Case-Based Pedagogical Reasoning Engine

Programmed by: Dianna Francesca M. Pardilla, Justin D. Vergara
Course & Section: CS0065 - AN41

[1] Run Default Assessment Scenario
[2] Enter Custom Student Error (Solve & Retain)
[3] View Entire Case Base Repository (5 cases)
[4] Exit System

Select option [1-4]: 1

▶ STEP 1: Learner Diagnosis Input [Ingestion]
-----
• Topic      : Algebra
• Detected Flaw : Formula Misuse
• Skill Level : Beginner
• Error Frequency : 5 repeated occurrences

▶ STEP 2: Experiential Retrieval [Retrieval Phase]
-----
• Best Match Case : CASE-002
• Match Affinity : ██████████ 79.0%
  ↳ Breakdown : {'Topic Match': 1.0, 'Error Match': 1.0, 'Level Match': 0.0, 'Freq Proximity': 0.6}
• Past Baseline : "Step-by-step formula breakdown focusing on variable substitution."

▶ STEP 3: Solution Adaptation & Revision [Pedagogical Tuning]
-----
✓ Action Assigned : Step-by-step formula breakdown focusing on variable substitution.
✓ Practice Set    : 3 guided formula substitution problems + 2 scaffolded verification drills
✓ Dynamic Pacing  : Decrease difficulty (Remedial Mode Activated)

▶ STEP 4: Experiential Retention [Continuous Learning]
-----
✦ Learning State : Preserving experience into local case memory...
✦ Stored Key     : CASE-006
✦ Memory Pool    : Case Base incremented to 6 active entries
```

INTELLIGENT TUTORING SYSTEM (ITS)
Case-Based Pedagogical Reasoning Engine

Programmed by: Dianna Francesca M. Pardilla, Justin D. Vergara
Course & Section: CS0065 - AN41

- [1] Run Default Assessment Scenario
- [2] Enter Custom Student Error (Solve & Retain)
- [3] View Entire Case Base Repository (6 cases)
- [4] Exit System

Select option [1-4]: 2

--- Enter New Student Data ---

Enter Topic (e.g., Algebra, Geometry, Calculus): Geometry
Enter Error Type (e.g., Formula Misuse, Sign Error, Unit Mismatch): Unit Mismatch
Proficiency Levels: [1] Beginner [2] Intermediate [3] Advanced
Select level (1-3): 3
Enter Error Frequency (1-10): 10

► **STEP 1: Learner Diagnosis Input** [Ingestion]

- Topic : Geometry
- Detected Flaw : Unit Mismatch
- Skill Level : Advanced
- Error Frequency : 10 repeated occurrences

► **STEP 2: Experiential Retrieval** [Retrieval Phase]

- Best Match Case : CASE-004
- Match Affinity : 70.0%
 - └ Breakdown : {'Topic Match': 1.0, 'Error Match': 1.0, 'Level Match': 0.0, 'Freq Proximity': 0.0}
- Past Baseline : "Unit conversion refresher focusing on metric dimension alignment."

► **STEP 3: Solution Adaptation & Revision** [Pedagogical Tuning]

- ✓ Action Assigned : Unit conversion refresher focusing on metric dimension alignment.
- ✓ Practice Set : 6 dimensional analysis drills + 2 scaffolded verification drills
- ✓ Dynamic Pacing : Decrease difficulty (Remedial Mode Activated)

► **STEP 4: Experiential Retention** [Continuous Learning]

- ✦ Learning State : Preserving experience into local case memory...
- ✦ Stored Key : CASE-007
- ✦ Memory Pool : Case Base incremented to 7 active entries

INTELLIGENT TUTORING SYSTEM (ITS)
Case-Based Pedagogical Reasoning Engine

Programmed by: Dianna Francesca M. Pardilla, Justin D. Vergara
Course & Section: CS0065 - AN41

- [1] Run Default Assessment Scenario
- [2] Enter Custom Student Error (Solve & Retain)
- [3] View Entire Case Base Repository (7 cases)
- [4] Exit System

Select option [1-4]: 3

CURRENT ITS CASE BASE REPOSITORY

- [CASE-001] Algebra | Sign Error (Beginner, Freq: 4)
Action : Review integer sign rules with interactive number-line drills.
Practice: 5 foundational integer arithmetic exercises
Pacing : Decrease difficulty
- [CASE-002] Algebra | Formula Misuse (Intermediate, Freq: 3)
Action : Step-by-step formula breakdown focusing on variable substitution.
Practice: 3 guided formula substitution problems
Pacing : Maintain current difficulty
- [CASE-003] Calculus | Chain Rule Omission (Advanced, Freq: 2)
Action : Visual breakdown of nested functions and outer/inner derivative scaffolding.
Practice: 4 multi-step chain rule problems
Pacing : Maintain current difficulty
- [CASE-004] Geometry | Unit Mismatch (Beginner, Freq: 5)
Action : Unit conversion refresher focusing on metric dimension alignment.
Practice: 6 dimensional analysis drills
Pacing : Decrease difficulty
- [CASE-005] Algebra | Order of Operations (Intermediate, Freq: 2)
Action : PEMDAS sequence reinforcement using color-coded grouping brackets.
Practice: 4 precedence-focused evaluations
Pacing : Maintain current difficulty
- [CASE-006] Algebra | Formula Misuse (Beginner, Freq: 5)
Action : Step-by-step formula breakdown focusing on variable substitution.
Practice: 3 guided formula substitution problems + 2 scaffolded verification drills
Pacing : Decrease difficulty (Remedial Mode Activated)

[CASE-007] Geometry | Unit Mismatch (Advanced, Freq: 10)

Action : Unit conversion refresher focusing on metric dimension alignment.

Practice: 6 dimensional analysis drills + 2 scaffolded verification drills

Pacing : Decrease difficulty (Remedial Mode Activated)

Press Enter to return to main menu...

INTELLIGENT TUTORING SYSTEM (ITS)
Case-Based Pedagogical Reasoning Engine

Programmed by: Dianna Francesca M. Pardilla, Justin D. Vergara
Course & Section: CS0065 - AN41

- [1] Run Default Assessment Scenario
- [2] Enter Custom Student Error (Solve & Retain)
- [3] View Entire Case Base Repository (7 cases)
- [4] Exit System

Select option [1-4]: 4

Exiting Intelligent Tutoring System. Goodbye!

V. REFLECTION

For this activity, we decided to divide our tasks so we could get things done faster. Dianna was responsible for writing, testing, and making the Python code, as well as capturing all the terminal output screenshots for our paper. Justin took care of putting together the overall documentation, organizing the sections, and detailing the system design. It wasn't completely smooth sailing though, because our laptop kept crashing in the middle of our runs, and our internet kept acting up and disconnecting, which made testing the code and sharing files super frustrating.

Despite those challenges, working on this lab made Case-Based Reasoning finally make sense to us. We realized that CBR doesn't just rely on rigid, hardcoded rules; instead, it actually mimics how people learn and solve problems by looking back at past scenarios. Seeing the system retrieve an old student case, tweak the lesson to fit the new mistake count, and immediately save that new experience into memory showed us how intelligent systems can continuously learn without starting from scratch.